Complete Execution Traces Impair SFT-Based Verdict Prediction: An Information Density Analysis

Anonymous ACL submission

Abstract

Complete execution traces contain all information needed for verdict prediction (classifying whether a test passes or fails), yet they actively impair SFT learning. On LLM-generated Python code, across four models spanning three architecture families (1.5B–13B), complete traces achieve only 72–82% accuracy despite containing strictly more diagnostic information than compact exception events (≥99.33%), an 18+pp degradation from *more* input. Training dynamics reveal the mechanism: verbose traces cause models to overfit to spurious patterns. `full_trace` with label-only loss (gradient dilution eliminated) oscillates between 82–89% across all training steps without stable convergence, while `exception_only` converges within 50 steps. Decomposition analysis on the Qwen family attributes at most ∼26% of the gap to gradient dilution (upper bound) and at least ∼74% to information overload at inference; a sharp performance jump at ∼500 tail tokens across all four models confirms that retaining diagnostic content, not volume, is the critical factor. A two-tier signal hierarchy holds cross-family: Tier 1 (exceptions, assertions; ≥98.53%) and Tier 2 (loop compression; 77–86%) are separated by 12+pp with non-overlapping confidence intervals. These results demonstrate that for SFT-based verdict prediction, more trace information impairs learning efficiency; the binding constraint is information density, not quantity.

1 Introduction

Execution traces (line-by-line records of a program's runtime behavior) have emerged as a promising input modality for code language models (Chen et al., 2025; Ding et al., 2024; Wang et al., 2025a; Ni et al., 2024). The intuition is straightforward: more execution information should help the model make better predictions. Yet for verdict prediction (classifying whether a test passes or fails), this intuition is wrong. Complete traces *actively degrade* accuracy by 18+pp compared to compact exception events, despite containing strictly more diagnostic information. *Why does more information impair SFT learning?*

Prior work found no single trace representation consistently outperforms others (Wang et al., 2025a), but focused on generation-oriented tasks. Verdict prediction offers a cleaner testbed: as binary classification, it isolates the learning signal from generation noise, enabling controlled component ablation that directly measures how trace content affects SFT learning efficiency.

We present the first systematic component ablation of trace representations for verdict prediction on LLM-generated Python code. Using 591 problems from HumanEval (Chen et al., 2021) and MBPP-sanitized (Austin et al., 2021), we decompose trace representations into individual components across four models spanning 1.5B to 13B parameters and three architecture families: Qwen-2.5-Coder-7B/1.5B-Instruct (Team, 2024), CodeLlama-13B-Instruct (Rozière et al., 2024), and DeepSeek-Coder-6.7B-Instruct (Guo et al., 2024).

The central result: complete traces (72–82% across four models) underperform compact exception events (≥99.33%) by 18+pp despite containing all diagnostic information, directly demonstrating that more information impairs SFT learning efficiency. Training dynamics confirm the mechanism: `exception_only` converges within 50 steps, while `full_trace` with label-only loss (gradient dilution eliminated) oscillates (82–89%) throughout training without reaching stable accuracy (Figure 3).

The degradation approximately decomposes (Qwen family, 2 scales) into information overload at inference (≥74%, lower bound) and gradient dilution (≤26%, upper bound). A tail-truncation budget sweep across all four models reveals a

sharp performance jump at ∼500 tokens, matching exception traceback length, confirming that retaining diagnostic content is the critical factor. Even on silent logic errors (55.5% of failures), exception events maintain 100% recall because test assertions generate exception events.

Our contributions:

1. **Complete traces actively impair SFT learning**: training dynamics directly show the mechanism: verbose traces cause overfitting to spurious patterns, with `full_trace` oscillating (82–89%) without stable convergence even when gradient dilution is eliminated. Seed variance negatively correlates with input density (σ=0.00 for `exception_only` to 4.48 for `no_trace`), and SFT closes a 6–49pp zero-shot gap across all models (§3.2, Appendix N).

2. **Exception events are the minimal sufficient signal**: $\geq$99.33% across all four models and all seeds tested (1.5B–13B, three families), with ∼91% semantic rather than format-driven (§3.1, §3.3). Supplementary OOD analysis on DebugBench (N=426) preserves the ranking, though margins narrow due to ceiling effects (§3.5).

3. **Approximate decomposition of the full-trace degradation**: on the Qwen family (7B and 1.5B, 3 seeds each), information overload accounts for at least ∼74% (lower bound) and gradient dilution at most ∼26% (upper bound), implying content selection matters more than loss engineering. A budget sweep across four models reveals a sharp performance jump at ∼500 tail tokens (§3.4, §3.5).

4. **Two-tier diagnostic hierarchy validated across four models and three architecture families**: Tier 1 signals (exceptions, assertions) achieve $\geq$98.53% across all four models with non-overlapping CIs against Tier 2; Tier 2 (loop compression, 77–86%) is model-dependent: Qwen +10.9/+7.2pp (7B/1.5B), DeepSeek +7.0pp, CodeLlama +0.9pp (§3.2).

2 Method

2.1 Task Definition and Information Density

We study *verdict prediction*: given a Python function, a test input, and an optional execution trace, predict whether the test passes or fails. The model produces a single token (`pass` or `fail`), making this a binary classification task. Unlike code generation (Chen et al., 2025) or output prediction (Gu et al., 2024), verdict prediction isolates the model's ability to detect faults from its ability to simulate execution, providing cleaner ablation contrasts than generation tasks.

We define *information density* as the ratio of verdict-diagnostic tokens to total sequence length, operationalized as the accuracy improvement per input token across trace conditions. `exception_only` inputs (∼150 tokens, 99.83%) exemplify high density; `full_trace` inputs (>2,048 tokens, ∼81%) exemplify low density despite containing strictly more diagnostic information.

2.2 Dataset

We sampled 10 candidate solutions per problem from Qwen-2.5-Coder-7B-Instruct (temperature 0.8, top_p 0.95) for 591 problems from HumanEval (Chen et al., 2021) (164) and MBPP-sanitized (Austin et al., 2021) (427), executed each against canonical tests with `sys.settrace` to obtain line-level traces and pass/fail verdicts, yielding 5,910 pairs (67% pass, 33% fail). The dataset is split 80/20 by problem ID into 4,720 training and 1,190 test samples (majority baseline: 71.51%). The `exception_only` format collapses passing solutions into identical rows (39.45% train, 48.99% test duplicates); these duplicates arise from multiple candidate solutions per problem, with the train–test split enforced at the problem level (472 train / 119 test problems, zero crossover; Appendix O).

2.3 Trace Representations

We define seven conditions that vary the trace information available (Figure 1):

- `no_trace`: Source code and test input only.
- `full_trace`: Complete line-by-line trace with variable states.
- `VOTA` (Verdict-Optimized Trace Abstraction): Exception events + assertion-point

comparisons + loop compression (Appendix C).

- `exception_only`: Only exception/error events.
- `ae_only`: Only assertion-point actual/expected pairs.
- `loop_only`: Only loop compression summaries.
- `VOTA_no_ae`: VOTA without assertion-point information.

2.4 Models and Training

We evaluate four models spanning three architecture families: **Qwen-2.5-Coder-7B/1.5B-Instruct** (Team, 2024), **CodeLlama-13B-Instruct** (Rozière et al., 2024), and **DeepSeek-Coder-6.7B-Instruct** (Guo et al., 2024), covering a 1.5B–13B parameter range (8.7×). All conditions use LoRA (Hu et al., 2022) (rank=16, α=32, dropout=0.05) on all linear layers, 3 epochs, effective batch size 32, LR 2×10^{-4} with linear warmup/decay, max sequence length 2,048 tokens. Sequences exceeding this limit are truncated while preserving the verdict label. The primary seed is 42; seeds 123 and 456 test `exception_only` robustness (Appendix B).

2.5 Loss Ablation

To disentangle training signal from input signal quality, we introduce `full_trace_label_only`: identical full-trace input but loss computed only on the verdict token (trace positions masked). Improvement over standard `full_trace` is attributable to *gradient dilution*; the remaining gap to VOTA reflects *information overload* at inference time.

3 Experiments

We evaluate across two research questions: (1) Why do complete traces degrade SFT verdict accuracy despite containing all diagnostic information? (2) Which trace components carry the verdict signal, and how does their density affect learning? All experiments use N=1,190 test samples; accuracy is reported with Wilson 95% CIs.[1]

[1]Data is partitioned at the problem level (472 train / 119 test problems, zero crossover; §2.2). The `exception_only` format collapses passing solutions into identical rows (48.99% test duplicates; Appendix O), but these arise from within-problem variants, not cross-set leakage; deduplication sensitivity analysis confirms 0.00pp delta across all 19 experimental conditions.

Table 1: Cross-model verdict accuracy (%, N=1,190) across four models (1.5B–13B) and three architecture families. Multi-seed entries report mean±std over 2–3 seeds; † = single seed (seed 42). Bold: best per model. Full per-seed breakdowns in Appendix E; per-class metrics in Appendix A.

Condition	Qwen-7B	Qwen-1.5B	CL-13B	DS-6.7B
exception_only	**99.83**±0.00	**99.83**±0.00	**99.89**±0.10	**99.39**±0.10
ae_only	99.64±0.49	98.83±0.36	99.08±0.00	98.53±0.30
VOTA	99.05±0.38	99.54±0.06	98.74±0.24	99.25‡
loop_only	86.05±0.52	79.54±1.42	77.40±1.54	80.31±1.65
no_trace	75.13†	72.35±4.48	76.47†	73.36†
Majority baseline	71.51			

DeepSeek `exception_only` 3 seeds: 99.33/99.50/99.33 (σ=0.10pp). VOTA on Qwen-7B: 99.41/98.66/99.08 (σ=0.38). `ae_only` 3 seeds: 99.92/99.08/99.92 (mean 99.64%); Qwen-7B `loop_only` 3 seeds: 85.63/85.88/86.64 (mean 86.05%). Qwen-1.5B `exception_only`: 99.83% across 3 seeds (σ=0). ‡DeepSeek VOTA 3 seeds: 84.45/99.16/99.33; seed 42 (84.45%, fail recall = 45.4%) is a training failure due to tokenizer-driven truncation (Appendix F), reported as 2-seed mean.

Pairwise comparisons use paired bootstrap resampling (B=10,000, seed=42); all differences are significant at $p<.0001$ unless noted.

3.1 Cross-Model Comparison

Table 1 compares four models spanning an 8.7× parameter range (1.5B–13B) and three architecture families (Qwen, DeepSeek, CodeLlama) under key conditions.

Exception dominance across families. Under `exception_only`, all four models achieve ≥99.33% across all seeds: Qwen-7B (99.83%±0.00, 3 seeds), Qwen-1.5B (99.83%), CodeLlama-13B (99.89%±0.10, 3 seeds; seed 123 achieves 100%), and DeepSeek-6.7B (99.39%±0.10, 3 seeds). The cross-model range is 0.50pp (99.39–99.89%), the tightest of any condition.

Exception signals across model scales. Exception-based accuracy is shared by models spanning 1.5B to 13B parameters (8.7×): both Qwen-1.5B and CodeLlama-13B achieve 99.83%+. CodeLlama-13B seed 123 achieves 100% (0/1190 errors), confirming that the 2 persistent errors (from `GeneratorExit`; Appendix G) are convergence artifacts rather than data limitations; 99.83% is a convergence floor, not a representation ceiling. In contrast,

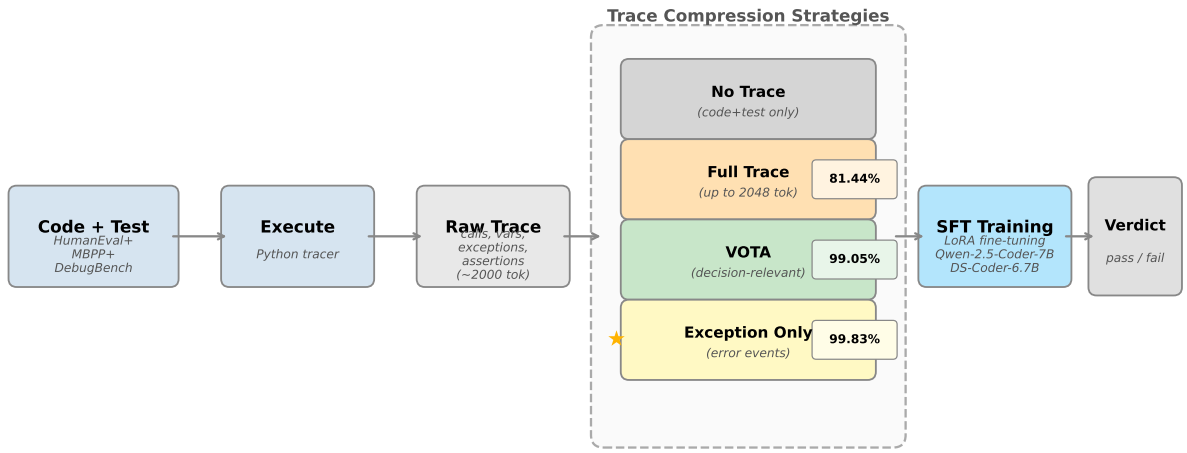


Figure 1: Overview of the verdict prediction pipeline. Source code and a test input are executed to produce a full raw trace. Four conditions vary the trace information: `no_trace` (75.13%), `full_trace` (81.44%), VOTA (99.05%), and `exception_only` (99.83%). Accuracies for Qwen-2.5-7B (N=1,190); multi-seed means where available (Table 2). Appendix D provides input/output examples.

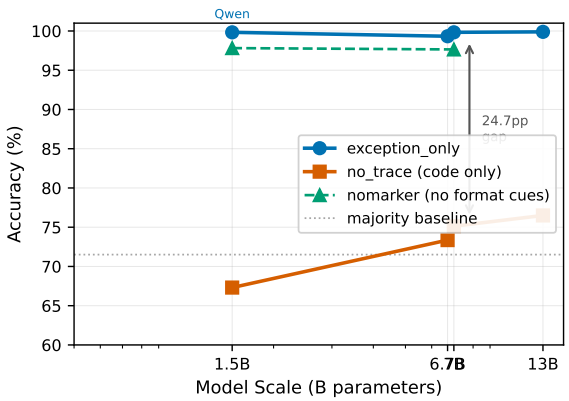


Figure 2: Verdict accuracy vs. model scale. `exception_only` maintains ≥99.33% regardless of scale, while `no_trace` shows monotonic sensitivity (+4.12pp). `nomarker` (markers removed) shows no scale degradation.

`no_trace` scales with capacity (72.35–76.47%, Figure 2) and shows the highest seed variance (σ=4.48pp vs. ≤1.65pp for trace conditions).

Scale robustness across tiers. Tier 1 signals are scale-invariant: `exception_only` spans 0.50pp across all four models (99.33–99.89%) and `ae_only` 1.18pp (98.74–99.92%; Table 1). Scale effects concentrate in Tier 2: `loop_only` spans 8.57pp (77.06–85.63%), with CodeLlama-13B (77.06%) below Qwen-1.5B (79.54%), suggesting architecture and pretraining matter more than parameter count for lower-density signals.

VOTA cross-family variation. Excluding outlier seeds, VOTA achieves ≥98.57% across all four models (0.80pp range). DeepSeek seed 42 (84.45%) is a training failure driven by tokenizer-induced truncation (91.7% of samples affected; Appendix F); `exception_only`'s compact inputs (median 150 tokens) sidestep truncation entirely.

3.2 Component Ablation

Multi-seed `ae_only` ablations use Qwen-7B; `loop_only` is evaluated across three models to test Tier 2 generalization (Table 1). Table 2 decomposes VOTA on Qwen-2.5-7B and reveals a two-tier hierarchy validated across all four models and three architecture families (Table 1): Tier 1 (≥98.53%) and Tier 2 (77–86%) are separated by 12+pp with non-overlapping CIs.

Tier 1 signals are individually sufficient: `exception_only` (99.83%±0.00) exceeds VOTA (99.05%±0.38) by +0.78pp (2/3 seeds p<.05, all directionally consistent), implying that combining signals introduces noise. Removing assertion points has no effect (`VOTA_no_ae` = 99.41%, single seed), confirming exception dominance. The Tier 1/2 boundary is unambiguous: the lowest Tier 1 condition (`ae_only` on DeepSeek, 98.53%±0.30, 2 seeds) exceeds the highest Tier 2 (`loop_only` on Qwen-7B, 86.05%, CI [83.97, 87.96]) by 12.48 pp with non-overlapping confidence intervals. Tier 2 `loop_only` lifts over `no_trace` are model-

Table 2: Component ablation on Qwen-2.5-7B (N=1,190). Tier 1 signals achieve near-perfect accuracy; combining them (VOTA) slightly dilutes performance. Entries marked †/§/‡ are 3-seed mean±std; unmarked entries are single seed (seed 42). Per-seed breakdowns in Appendix E. Seed σ negatively correlates with density (0.00 for exception to 0.52 for loop).

Condition	Components	Acc (%)	95% CI	Tier
ae_only	assertion-point	99.64†	[99.26, 99.87]	1
exception_only	exception events	**99.83**	[99.58, 100.00]	1
VOTA	exc+ae+loop	99.05§	[98.49, 99.50]	—
VOTA_no_ae	exc+loop	99.41	[98.90, 99.80]	—
loop_only	loop compression	86.05‡	[83.97, 87.96]	2
full_trace	complete trace	81.44±1.08	—	—
no_trace	none	75.13	[72.77, 77.48]	—
Regex (best)	exc. type regex	93.61	—	—
3-shot (Qwen-7B)	in-context learning	90.84	—	—
Zero-shot (4 models)	prompting	51.0–93.5	—	—

dependent: Qwen shows +10.92pp (7B) and +7.19pp (1.5B, $p<.0001$), with the larger 7B gain likely reflecting greater capacity to exploit structured loop patterns; DeepSeek +6.95pp (80.31%), CodeLlama +0.93pp (77.40%, n.s.).

Rule-based baseline comparison. The strongest non-trivial regex heuristic (B4: exception class name matching; Table 19) achieves 93.61% accuracy (F1=0.876), trailing SFT `exception_only` (99.83%, F1=0.997) by 6.22pp (McNemar $p<.001$). The gap concentrates on caught-exception boundary cases: regex flags all exception events as failures, while SFT learns to distinguish caught exceptions (benign, inside `try/except`) from uncaught faults, correctly resolving 6/8 boundary samples (Appendix P). Across all formats, SFT outperforms the best applicable regex by +3.62–23.86pp (Appendix P).

Zero-shot and few-shot LLM baselines. Zero-shot prompting spans 51.0–93.5% across four models (42.44pp range); three-shot ICL on Qwen-7B (90.84%) improves only 1.76pp. SFT compresses this range to 0.50pp (99.39–99.89%), closing a 6–49pp gap with 4,720 samples (Appendix Q).

Frontier model crossover. Two frontier models confirm that SFT's advantage depends on input density. GPT-4o zero-shot achieves 87.98% on `exception_only` and 84.45% on `no_trace`; GPT-4.1 achieves 82.18% and 77.31%, respectively. Under `no_trace`, both frontier models outperform SFT Qwen-7B (75.13%) by +9.32pp (GPT-4o) and +2.18pp (GPT-4.1). Under `exception_only`, the ranking reverses: SFT Qwen-7B (99.83%) exceeds GPT-4o by 11.85pp and GPT-4.1 by 17.65pp. This crossover shows that compact diagnostic input unlocks SFT's full advantage, while absent or verbose traces favor general reasoning capacity.

Table 3: Approximate information density decomposition: gradient dilution (∼26%) vs. information overload (∼74%). All entries are 3-seed means on Qwen-7B.

Condition	Acc (%)	Δ vs. full_trace	Gap explained
full_trace	81.44±1.08	—	—
label_only	85.94±1.12	+4.50pp	26%
VOTA	99.05±0.38	+17.61pp	100%

3.3 Format Marker Ablation

The `exception_only` format includes a `[No exception data in trace]` marker in all pass-class samples (regex baseline B5 exploits this for 99.33% accuracy; Appendix P). To quantify the marker's contribution, we retrain with all format markers removed (`nomarker` condition).

Qwen-7B 3-seed mean: 97.65% ± 0.96 (seeds 42/123/456: 96.55/98.32/98.07%), a drop of 2.18pp from `exception_only` (99.83%). Qwen-1.5B: 97.82%, drop of 2.01pp. The marker contribution (∼2pp) accounts for only ∼9% of the total exception lift over `no_trace` (24.70pp on 7B). Regex baselines confirm this decomposition: B5 (`[No exception data]`→pass, 99.33% with markers) collapses to 28.49% without markers, confirming it is entirely format-dependent. The strongest semantic regex B4 (exception class matching, 93.61%) is marker-invariant; SFT `nomarker` (97.65%) exceeds it by 4.04pp, isolating the learned semantic contribution beyond pattern matching.

3.4 Information Density Decomposition

The 17.61pp gap between `full_trace` (81.44%±1.08, 3-seed mean) and VOTA (99.05%, 3-seed mean) decomposes via `full_trace_label_only` (85.94%±1.12, 3 seeds; Table 3), which retains the full trace but masks all trace tokens from the loss.

Gradient dilution accounts for at most ∼26% of the gap (upper bound; +4.50pp, 3-seed mean): when the model no longer wastes gradient updates on predicting trace tokens, accuracy improves even with identical input. The remaining at least ∼74% (+13.11pp, lower bound) reflects information overload at inference: the model re-

ceives thousands of irrelevant trace lines that compete for attention with the diagnostic signal, regardless of loss design. This asymmetry implies that content selection matters more than loss engineering. Cross-model evidence supports this: the tail-truncation sharp performance jump at ∼500 tokens (§3.5, all four models) coincides with exception traceback length, indicating that retaining diagnostic content is the critical factor. Cross-scale validation on Qwen-1.5B yields a consistent pattern (17% gradient dilution, 83% information overload, 3 seeds), confirming that information overload dominates across model scales. CodeLlama-13B and DeepSeek-6.7B both exhibit label-only collapse (28.5% accuracy, collapsing to the minority class), limiting the decomposition to models whose tokenizers preserve sufficient trace structure under the training sequence length. Error analysis provides direct evidence: all 217 `full_trace` errors occur in samples where tokenizer truncation removes diagnostic markers, and among the 61 samples where `full_trace` underperforms `no_trace`, all exhibit the same truncation pattern, confirming that information overload manifests through a mechanical truncation pathway rather than semantic confusion. The two factors may interact, and label-only loss simultaneously changes multiple optimization dynamics; the 26%/74% attribution is an approximate decomposition (Qwen family, 2 scales, 3 seeds each).

3.5 Additional Analyses

Trace length ablation. Varying the head-truncated trace token budget (256–no limit) on Qwen-7B yields accuracy in a narrow 77.2%–81.8% band with a non-monotonic pattern: 1024-token prefixes (77.23%) underperform 512-token prefixes (80.42%, p=.002), because mid-length prefixes retain enough irrelevant content to mislead the model while discarding diagnostic events near the trace terminus (Appendix I). This non-monotonicity is specific to head-truncation; under tail-truncation, accuracy increases monotonically for two of three models (Table 12 in Appendix), with only DeepSeek showing a mild architecture-specific reversal at tail-1024→tail-2048 (−1.5pp).

Truncation direction and budget curve. Tail-truncation (retaining the last k tokens) dramatically outperforms head-truncation: among the three models with complete data, tail-2048 exceeds head-2048 by 15.5–22.6pp, with the gap widening at smaller scale (Qwen-1.5B: +22.6pp vs. Qwen-7B: +15.5pp; Appendix I, Table 13). All four models' head-2048 accuracies converge into a narrow 75.3–81.8% band (6.5pp), comparable to the tail-256 band (72.6–78.9%, 6.3pp), indicating that signal-poor conditions produce a shared performance floor regardless of scale or architecture.

A cross-model budget sweep across four models and three architecture families reveals a sharp performance jump localized between 384 and 512 tail tokens. At tail-384, all models remain near baseline (73.6–82.9%, only +1.0–6.5pp over tail-256); the steep jump occurs at 384→512 (+11.0–22.0pp), crossing from near-baseline to near-ceiling (∼94–98%). This localizes the critical threshold to ∼400–500 tokens, matching the typical exception traceback length (Table 12). Beyond 512 tokens, accuracy converges further: multi-seed validation across all 12 model×seed combinations confirms all tail-512 accuracies fall within a 4.63pp band [93.61%, 98.24%] (Table 14); at tail-1024, three models reach 96.5–98.2%, compressing the 6.3pp cross-model spread observed at tail-256. `tail_2048` captures 95–98% of `exception_only`'s accuracy across all completed configurations. The residual 2–5pp gap confirms that semantic extraction (identifying exception type and message) still outperforms positional extraction (retaining a fixed window containing non-diagnostic lines).

Calibration. Calibration mirrors the tier hierarchy: `exception_only` achieves ECE = 0.0027 vs. 0.171 for `no_trace`, a 63× gap (Figure 5). This calibration advantage generalizes across architectures and scales: all four models (1.5B–13B) achieve ECE $<$ 0.005 under `exception_only` (Table 22 in Appendix), indicating well-calibrated predictions rather than overfit confidence.

Error analysis. `exception_only`'s 2 errors have zero overlap with all other conditions' error sets (Jaccard = 0.000; Appendix K); per-type breakdown confirms all standard exception types achieve 100% accuracy, with only `GeneratorExit` (a benign `BaseException` subclass) causing failures (Appendix H).

Length control. Padding `exception_only` to 2,048 tokens, even in a semantically loaded position (after the verdict region), yields identical accuracy (99.83%), confirming the 18+pp gap arises

from spurious patterns in verbose trace content, not sequence length.

Supplementary OOD analysis. On DebugBench (Tian et al., 2024) (N=426), the condition ranking is preserved: `exception_only` 100%, VOTA 99.77%, `no_trace` 96.71%. However, narrow margins (3.29pp headroom from `no_trace`) limit discriminative power; this supplementary analysis confirms the ranking but cannot establish effect sizes comparable to the primary dataset (Appendix L).

4 Discussion

The central finding is that more trace information actively impairs SFT learning for verdict prediction. Exception events concentrate the runtime's primary fault signal (error type, stack frame, and message) in a compact format (median 150 tokens), while non-exception components (variable states, loop iterations, control flow) record *how* the program ran, not *whether* it failed.

This density perspective explains three key results: (1) adding trace information beyond exceptions provides no benefit (Table 2), because low-density content competes for attention; (2) a tail-truncation budget sweep reveals a sharp performance jump at ∼500 tokens (Table 12), matching the boundary where diagnostic events dominate; (3) the SFT advantage over regex concentrates on caught-exception boundary cases, confirming semantic signal extraction (§3.2). The narrow cross-model gap (Table 1) supports that signal density, not model capacity, is the binding constraint.

Why exception_only outperforms ae_only. Although `ae_only` is a strict superset, it underperforms `exception_only` (Table 2), indicating extra content introduces noise rather than signal.

Why SFT over regex? Two regex baselines bracket the format-marker contribution: B_error (`[ERROR]`→fail) achieves 93.45% and B_marker (`[No exception data]`→pass) achieves 99.33% (Appendix P). SFT `exception_only` (99.83%) exceeds B_marker by 0.50pp; the gap concentrates on 6 caught-exception samples (e.g., `check_dict_case` in HumanEval/95, where `try/except` catches a `ValueError` so the trace contains `[ERROR]` yet the test passes). Regex flags all such samples as failures; SFT correctly distinguishes caught from uncaught exceptions (McNemar p=.014). Beyond boundary accuracy, SFT provides calibrated probabilities (ECE = 0.0027 vs. binary regex; §3.5) and format robustness: under `nomarker`, B5 collapses to 28.49% while SFT retains 97.65% (+4.04pp over the strongest semantic regex B4; §3.3).

Prompting baselines and error floor. Zero-shot spans a 42pp range; SFT compresses it to sub-1pp (§3.5). The residual 2/1190 errors (both `GeneratorExit`; Appendix G) are shared across configurations; CodeLlama-13B seed 123 achieves 100%, confirming 99.83% is a convergence *floor*.

Tier hierarchy and methodological implications. The two-tier structure (Table 1) suggests a fallback design: extract exception events first, resort to Tier 2 only when no exception is raised. Table 1 confirms `exception_only` dominance across all four models; Tier 2 benefit is model-dependent (Qwen +10.9/+7.2pp at 7B/1.5B; CodeLlama +0.9pp, possibly reflecting limited headroom from the 76.47% no-trace baseline). Whether the density effect transfers to other trace-based tasks (Gu et al., 2024; Ni et al., 2024) remains open.

Caveats on the density interpretation. Tier 1 cross-model range is <1pp vs. ∼9pp for Tier 2, but N=4 models is insufficient to establish a continuous relationship, and density is confounded with signal type. The decomposition (§3.4) is approximate: `label_only` simultaneously alters gradient source distribution, effective learning rate, and attention allocation; padding `exception_only` to 2,048 tokens preserves accuracy (§3.5), controlling for the sequence length confound. CodeLlama and DeepSeek both exhibit label-only collapse (28.5%), preventing cross-family replication of the decomposition.

Exception events capture test verdict signals, not just crashes. Although 55.5% of failures are silent logic errors, `exception_only` achieves 100% fail recall because test assertions generate `AssertionError` events, confirming "exception events" encompasses both crashes and assertion failures.

Information density and learning dynamics. Figure 3 quantifies the contrast: `exception_only` reaches 99%+ at step 50, while `full_trace` with label-only loss oscillates (82–89%) across all 444 steps, a signature

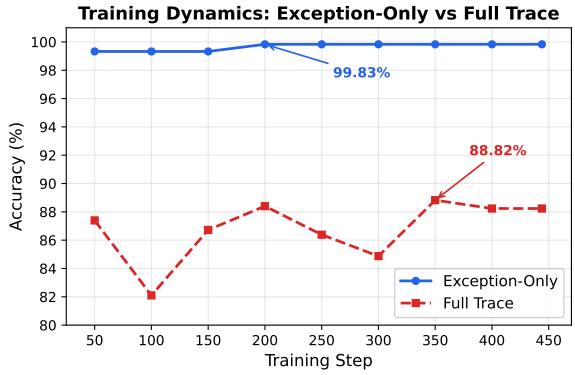


Figure 3: Training dynamics: `exception_only` converges to 99.83% by step 150; `full_trace` with label-only loss oscillates (82–89%) without convergence.

of overfitting to spurious patterns (Appendix N). Seed variance negatively correlates with input density (Table 2).

5 Related Work

Execution Traces for Code LLMs. TracePile (Chen et al., 2025) scaled trace-augmented training to 2.6M samples; Wang et al. (2025a) found trace utility task- and format-dependent, which we extend by identifying exception events as the dominant signal. NExT (Ni et al., 2024) and ET-CoT (Eguchi and Akiba, 2025) used trace chain-of-thought for repair and execution reasoning; Armengol-Estape et al. (2025) found compact traces comparable to full traces, supporting our density finding. Liu et al. (2023) pre-trained on execution traces; CRUXEval (Gu et al., 2024), extended to 19 languages (Xu et al., 2025), established execution reasoning benchmarks. None decompose which trace components contribute to *classification* accuracy.

Code Verdict and Patch Classification. TRACED (Ding et al., 2024) showed execution-aware pretraining improves vulnerability detection; DeepDFA (Steenhoek et al., 2024) combined classical analysis with deep learning; CC2Vec (Hoang et al., 2020), LLM4PatchCorrect (Zhou et al., 2024), and APPT (Zhang et al., 2024) developed representation learning of code changes. Verdict prediction is an instance of the test oracle problem (Barr et al., 2015); FuseFL (Widyasari et al., 2024) and Foster et al. (2025) demonstrated structured fault signals. None isolate individual trace components or compare against rule-based baselines.

Fault Localization. Spectrum-based fault localization (SBFL) ranks program elements by suspiciousness from test spectra (Abreu et al., 2007; Jones et al., 2002; Wong et al., 2014); recent LLM-based approaches (Kang et al., 2024; Qin et al., 2024) share our insight that anomalous execution events carry the strongest diagnostic signal, with verdict prediction serving as a prerequisite to FL.

Trace Compression and Selection. TraceCoder (Huang et al., 2026) and InspectCoder (Wang et al., 2025c) use selective trace extraction; RepairAgent (Bouzenia et al., 2025), Agentless (Xia et al., 2024), SWE-agent (Yang et al., 2024b), and OpenHands (Wang et al., 2025b) incorporate execution feedback in repair workflows within the SWE-bench ecosystem (Jimenez et al., 2024), yet which trace components carry verdict signals most efficiently remains unstudied. Prompt compression (Jiang et al., 2023; Li et al., 2023) showed pruning low-information tokens preserves accuracy; our approximate decomposition (at most $\sim$26% gradient dilution, at least $\sim$74% information overload; Qwen family) quantifies *why*.

Calibration and Defect Prediction. Neural network calibration (Guo et al., 2017), extended to LLMs (Kadavath et al., 2022; Geng et al., 2024; Spiess et al., 2025), studies confidence-correctness alignment; our ECE < 0.005 suggests compact inputs improve calibration. Just-in-time defect prediction (Kamei et al., 2013; Hoang et al., 2019) shares our fault detection motivation.

Training Efficiency and Data Selection. Curriculum learning (Bengio et al., 2009), dataset cartography (Swayamdipta et al., 2020), and data selection (Sachdeva et al., 2024; Xie et al., 2023) show that data quality improves convergence, consistent with our finding that compact exceptions converge rapidly while verbose traces oscillate.

6 Conclusion

Across four models (1.5B–13B, three families), complete traces (72–82%) underperform compact exception events ($\geq$99.33%) by 18+pp, with at least $\sim$74% of the gap attributable to information overload. A two-tier signal hierarchy (Tier 1 $\geq$98.53%, Tier 2 77–86%, 12+pp gap) holds cross-family, suggesting a fallback design: exception events first, loop summaries as secondary signal. Generalization beyond LLM-generated Python short functions remains open.

Limitations

Our study has several limitations. First, all experiments use a single dataset (591 problems from HumanEval and MBPP-sanitized) of LLM-generated Python short functions; while DebugBench (§3.5) confirms exception dominance on synthetically injected bugs, generalization to human-written code, other languages, and multi-file projects remains unverified. Second, all four models (1.5B–13B) are code-specialized; whether these findings extend to general-purpose LLMs or larger scales is untested. Third, our analysis is limited to verdict prediction; whether the information density advantage transfers to other trace-based tasks (e.g., output prediction, program repair) is an open question.

Ethics Statement

This work uses publicly available benchmarks (HumanEval, MBPP-sanitized, DebugBench) and does not involve human subjects, crowdsourced annotations, or personally identifiable information. All models evaluated are open-source and publicly accessible. Training uses parameter-efficient LoRA fine-tuning on enterprise-grade GPUs, with each experiment using a single GPU. Total compute is approximately 320 GPU-hours across all conditions, models, seeds, and iterative refinements (including approximately 170 hours for successful experiments; individual runs range from 0.5h for small models to 2.2h for CodeLlama-13B, with the total reflecting 4 models $\times$ multiple conditions $\times$ multiple seeds $\times$ iterative development). We do not foresee direct negative societal impacts from this work; verdict prediction is a developer-facing tool for automated testing.

References

Rui Abreu, Peter Zoeteweij, and Arjan J.C. van Gemund. 2007. On the accuracy of spectrum-based fault localization. *Testing: Academic and Industrial Conference Practice and Research Techniques (MUTATION)*.

Jordi Armengol-Estape, Quentin Carbonneaux, Tianjun Zhang, Aram H. Markosyan, Volker Seeker, Chris Cummins, Melanie Kambadur, Michael F.P. O'Boyle, Sida Wang, Gabriel Synnaeve, and Hugh Leather. 2025. What i cannot execute, i do not understand: Training and evaluating LLMs on program execution traces. *arXiv preprint arXiv:2503.05703*.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. 2015. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525.

Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. 2009. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning (ICML)*, pages 41–48.

Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2025. RepairAgent: An autonomous, LLM-based agent for program repair. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, and 1 others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Nuo Chen, Zehua Li, Keqin Bao, Junyang Lin, and Dayiheng Liu. 2025. Chain of execution supervision promotes general reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Yangruibo Ding, Ben Steenhoek, Kexin Pei, Gail Kaiser, Wei Le, and Baishakhi Ray. 2024. Traced: Execution-aware pre-training for source code. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*.

Koshi Eguchi and Takuya Akiba. 2025. Making llms program interpreters via execution trace chain of thought. In *ICML 2025 Workshop on Programmatic Representations for Agent Learning (PRAL)*.

Christopher Foster, Abhishek Gulati, Mark Harman, Inna Harper, Ke Mao, Jillian Ritchey, Hervé Robert, and Shubho Sengupta. 2025. Mutation-guided LLM-based test generation at meta. In *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE), Industry Track*.

Jiahui Geng, Fengyu Cai, Yuxia Wang, Heinz Koeppl, Preslav Nakov, and Iryna Gurevych. 2024. A survey of confidence estimation and calibration in large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*.

Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I. Wang. 2024. Cruxeval: A benchmark for code reasoning, understanding and execution. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1321–1330.

Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y.K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. Deepseek-coder: When the large language model meets programming – the rise of code intelligence. *arXiv preprint arXiv:2401.14196*.

Thong Hoang, Hoa Khanh Dam, Yasutaka Kamei, David Lo, and Naoyasu Ubayashi. 2019. DeepJIT: An end-to-end deep learning framework for just-in-time defect prediction. In *Proceedings of the 16th International Conference on Mining Software Repositories (MSR)*, pages 34–45.

Thong Hoang, Hong Jin Kang, Julia Lawall, and David Lo. 2020. CC2Vec: Distributed representations of code changes. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE)*, pages 518–529.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. Lora: Low-rank adaptation of large language models. In *Proceedings of the 10th International Conference on Learning Representations (ICLR)*.

Jiangping Huang, Wenguang Ye, Weisong Sun, Jian Zhang, Mingyue Zhang, and Yang Liu. 2026. TraceCoder: A trace-driven multi-agent framework for automated debugging of LLM-generated code. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE)*.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 13358–13376.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.

James A. Jones, Mary Jean Harrold, and John Stasko. 2002. Visualization of test information to assist fault localization. In *Proceedings of the 24th International Conference on Software Engineering (ICSE)*.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, and 17 others. 2022. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*.

Yasutaka Kamei, Emad Shihab, Bram Adams, Ahmed E. Hassan, Audris Mockus, Anand Sinha, and Naoyasu Ubayashi. 2013. A large-scale empirical study of just-in-time quality assurance. *IEEE Transactions on Software Engineering*, 39(6):757–773.

Sungmin Kang, Gabin An, and Shin Yoo. 2024. AutoFL: Automated fault localization via large language models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*.

Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. 2023. Compressing context to enhance inference efficiency of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Chenxiao Liu, Shuai Lu, Weizhu Chen, Daxin Jiang, Alexey Svyatkovskiy, Shengyu Fu, Neel Sundaresan, and Nan Duan. 2023. Code execution with pre-trained language models. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 4984–4999.

Ansong Ni, Miltiadis Allamanis, Arman Cohan, Yinlin Deng, Kensen Shi, Charles Sutton, and Pengcheng Yin. 2024. Next: Teaching large language models to reason about code execution. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.

Yihao Qin, Shangwen Wang, Yiling Lou, Jinhao Dong, Kaixin Wang, Xiaoling Li, and Xiaoguang Mao. 2024. AgentFL: Scaling llm-based fault localization to project-level context. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Rémi Lacroix, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, and 7 others. 2024. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*.

Noveen Sachdeva, Benjamin Coleman, Wang-Cheng Kang, Jianmo Ni, Lichan Hong, Ed H. Chi, James Caverlee, Julian McAuley, and Derek Zhiyuan Cheng. 2024. How to train data-efficient LLMs. *arXiv preprint arXiv:2402.09668*.

Claudio Spiess, David Gros, and 1 others. 2025. Calibration and correctness of language models for code. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*.

Benjamin Steenhoek, Hongyang Gao, and Wei Le. 2024. Dataflow analysis-inspired deep learning for efficient vulnerability detection. In *Proceedings of*

the 46th IEEE/ACM International Conference on Software Engineering (ICSE).

Swabha Swayamdipta, Roy Schwartz, Nicholas Lourie, Yizhong Wang, Hannaneh Hajishirzi, Noah A. Smith, and Yejin Choi. 2020. Dataset cartography: Mapping and diagnosing datasets with training dynamics. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 9275–9293.

Qwen Team. 2024. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186*.

Runchu Tian, Yining Ye, Yujia Qin, Xin Cong, Yankai Lin, Yinxu Pan, Yesai Wu, Haotian Hui, Weichuan Liu, Zhiyuan Liu, and Maosong Sun. 2024. DebugBench: Evaluating debugging capability of large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*.

Jian Wang, Xiaofei Xie, Qiang Hu, Shangqing Liu, and Yi Li. 2025a. Do code semantics help? a comprehensive study on execution trace-based information for code large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2025*.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Daniel Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, and 5 others. 2025b. OpenHands: An open platform for AI software developers as generalist agents. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*.

Yunkun Wang, Yue Zhang, Guochang Li, Chen Zhi, Binhua Li, Fei Huang, Yongbin Li, and Shuiguang Deng. 2025c. InspectCoder: Dynamic analysis-enabled self repair through interactive LLM-debugger collaboration. *arXiv preprint arXiv:2510.18327*.

Ratnadira Widyasari, Jia Wei Ang, Truong Giang Nguyen, Neil Sharma, and David Lo. 2024. Demystifying faulty code: Step-by-step reasoning for explainable fault localization. In *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*.

W. Eric Wong, Vidroha Debroy, Ruizhi Gao, and Yihao Li. 2014. DStar: An effective spectrum-based fault localization technique. *Journal of Systems and Software*, 91:44–58.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*.

Sang Michael Xie, Hieu Pham, Xuanyi Dong, Nan Du, Hanxiao Liu, Yifeng Lu, Percy Liang, Quoc V. Le, Tengyu Ma, and Adams Wei Yu. 2023. DoReMi: Optimizing data mixtures speeds up language model pretraining. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Ruiyang Xu and 1 others. 2025. CRUXEval-X: A benchmark for multilingual code reasoning, understanding and execution. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*.

Haoran Yang, Yumeng Zhang, Jiaqi Xu, Hongyuan Lu, Pheng Ann Heng, and Wai Lam. 2024a. Unveiling the generalization power of fine-tuned large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024b. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Quanjun Zhang, Chunrong Fang, Weisong Sun, Yan Liu, Tieke He, Xiaodong Hao, and Zhenyu Chen. 2024. APPT: Boosting automated patch correctness prediction via fine-tuning pre-trained models. *IEEE Transactions on Software Engineering*, 50(3).

Xin Zhou, Bowen Xu, Kisub Kim, DongGyun Han, Hung Huu Nguyen, Thanh Le-Cong, Junda He, Bach Le, and David Lo. 2024. Leveraging large language model for automatic patch correctness assessment. *IEEE Transactions on Software Engineering*, 50(11).

A Per-Class Performance Breakdown

Table 4 reports per-class precision, recall, and F1 for all experimental conditions.

B Training Details

Table 5 summarizes the training hyperparameters shared across all conditions. All experiments use the HuggingFace TRL `SFTTrainer` with LoRA (Hu et al., 2022) applied to all linear layers.

Table 6 reports per-condition training wall-clock times. Conditions with longer trace inputs (e.g., `full_trace`, VOTA) require proportionally more training time due to increased sequence lengths.

C VOTA Extraction Algorithm

Algorithm 1 describes the three-component VOTA extraction pipeline applied to a raw `sys.settrace` execution log.

Table 4: Per-class precision, recall, and F1 for all experimental conditions (seed 42). DeepSeek VOTA seed 42 is a training failure (84.45%, fail recall = 45.4%) due to tokenizer-driven truncation; seeds 123/456 achieve 99.16%/99.33%.

Condition	Model	Acc	Pass P	Pass R	Pass F1	Fail P	Fail R	Fail F1
No Trace	Qwen-2.5-7B	75.13	87.76	75.79	81.34	54.73	73.45	62.72
Full Trace	Qwen-2.5-7B	81.76	87.74	86.60	87.17	67.43	69.62	68.51
Full Trace (LO)	Qwen-2.5-7B	85.94	88.21	93.18	90.63	80.07	68.73	73.97
VOTA	Qwen-2.5-7B	99.05	100.00	99.18	99.59	97.98	100.00	98.98
Exception Only	Qwen-2.5-7B	99.83	100.00	99.76	99.88	99.41	100.00	99.71
AE Only	Qwen-2.5-7B	99.92	100.00	99.88	99.94	99.71	100.00	99.85
Loop Only	Qwen-2.5-7B	86.05	92.39	87.07	89.66	71.65	82.01	76.48
VOTA_no_ae	Qwen-2.5-7B	99.41	99.88	99.29	99.59	98.26	99.71	98.98
No Trace	DeepSeek-6.7B	73.36	81.71	80.85	81.28	53.16	54.57	53.86
Full Trace	DeepSeek-6.7B	79.16	85.70	85.10	85.40	63.20	64.30	63.70
VOTA	DeepSeek-6.7B	84.45	82.14	100.00	90.20	100.00	45.40	62.50
Exception Only	DeepSeek-6.7B	99.33	100.00	99.06	99.53	97.69	100.00	98.83
AE Only	DeepSeek-6.7B	98.74	98.40	99.90	99.10	99.70	95.90	97.70

Table 5: Training hyperparameters (shared across all conditions).

Hyperparameter	Value
LoRA rank (r)	16
LoRA alpha (α)	32
LoRA dropout	0.05
LoRA target modules	all linear layers
Epochs	3
Per-device batch size	4
Gradient accumulation steps	8
Effective batch size	32
Learning rate	2×10^{-4}
LR scheduler	linear decay
Warmup ratio	0.1
Weight decay	0.01
Max sequence length	2048 tokens
Primary seed	42
Robustness seeds	123, 456
Hardware	1× RTX PRO 6000 Blackwell (96 GB VRAM)
Framework	HuggingFace TRL SFTTrainer

Table 6: Training wall-clock time per condition on a single RTX PRO 6000 Blackwell GPU. Where multiple seeds were run, individual times are listed.

Condition	Seed(s)	Time (h)
no_trace	42	∼0.5
full_trace	42	∼2.2
label_only	42	∼2.2
VOTA	42	∼2.3
exception_only	42	∼2.1
exception_only	123	∼2.1
exception_only	456	∼1.3
ae_only	42	∼1.8
loop_only	42	∼2.1
VOTA_no_ae	42	∼1.8

D VOTA Trace Examples

We provide two representative VOTA trace examples illustrating the fail and pass cases. In the fail case, the exception event provides an unambiguous fault signal. In the pass case, the absence of an exception event combined with a matching assertion point confirms correct execution.

Fail example. A `ZeroDivisionError` triggers an `[ERROR]` event, providing a direct fault signal that is sufficient for the model to predict `fail`.

```
## Function
def divide(a, b):
    return a / b

## Test Input
divide(1, 0)

## Expected Output
inf

## VOTA Trace
[ERROR] ZeroDivisionError: division by zero

## Verdict
fail
```

Pass example. No exception is raised, and the assertion point confirms that the actual output matches the expected value.

```
## Function
def add(a, b):
    return a + b

## Test Input
add(2, 3)

## Expected Output
5

## VOTA Trace
[LOOP] No loops detected.
[ASSERT] actual=5, expected=5

## Verdict
pass
```

These examples illustrate the core asymmetry exploited by the `exception_only` condition: fail cases produce an explicit `[ERROR]` event, while pass cases are characterized by its absence. This binary signal (presence vs. absence of an exception) is sufficient for near-perfect verdict pre-

Algorithm 1 VOTA Trace Extraction

Require: Raw trace T from `sys.settrace` (list of events)
Ensure: Compressed VOTA representation V

$V \leftarrow$ empty string
// Component 1: Exception summary
for each event e in T with type `exception` **do**
  Parse traceback $\rightarrow$ extract error type, message, file, line
  Append `[ERROR] type: message (file:line)` to V
end for
if no exception events found **then**
  Append `[No exception data in trace]` to V
end if
// Component 2: Assertion-point comparison
for each `assert` or `return` event e in T **do**
  Extract actual value from execution state
  Extract expected value from test assertion
  Append `[ASSERT] actual=... expected=... match=T/F` to V
end for
// Component 3: Loop compression
for each consecutive repeated line sequence S in T **do**
  if $|S| \geq 2$ iterations of same loop body **then**
    Replace S with single iteration + `[repeated ×N]`
  end if
end for
Append compressed non-exception lines to V
return V

Table 7: Full results for Qwen-2.5-Coder-7B across all conditions and seeds. Conditions with a single seed report no variance. `exception_only` achieves 99.83% across all three seeds (σ=0). `ae_only` achieves 99.64% mean ($\pm$0.49; seed 123 at 99.08% is the outlier).

Condition	S42	S123	S456	Mean $\pm$ Std
exception_only	99.83	99.83	99.83	99.83 $\pm$ 0.00
ae_only	99.92	99.08	99.92	99.64 $\pm$ 0.49
VOTA	99.41	98.66	99.08	99.05 $\pm$ 0.38
VOTA_no_ae	99.41	—	—	—
label_only	86.22	86.89	84.70	85.94 $\pm$ 1.12
loop_only	85.63	85.88	86.64	86.05 $\pm$ 0.52
full_trace	81.76	80.20	82.35	81.44 $\pm$ 1.08
no_trace	75.13	—	—	—

Table 8: Full results for DeepSeek-Coder-6.7B across all conditions and seeds. `exception_only` is consistent across seeds: 99.33/99.50/99.33 (σ=0.10pp).

Condition	Seed	Accuracy	Notes
exception_only	42	99.33	epoch 1: 99.83
exception_only	456	99.33	CI [98.82, 99.75]
exception_only	123	99.50	
VOTA	42	84.45	training failure; fail recall = 45.4%
VOTA	123	99.16	
VOTA	456	99.33	
no_trace	42	73.36	
full_trace	42	79.16	CI [76.80, 81.30]

diction.

E Full Experimental Results

Tables 7 and 8 report the complete experimental results across all conditions, models, and seeds.

F Tokenizer and Truncation Analysis

DeepSeek VOTA seed 42 (84.45%) is a training failure driven by tokenizer-induced truncation. Re-evaluation of the seed 42 checkpoint with the standard max_length=2048 yields 16% accuracy, confirming complete model collapse rather than marginal degradation. The previously reported 84.45% was obtained with max_length=16384, which partially bypasses tokenizer truncation. Seeds 123 and 456, evaluated under identical conditions, achieve 99.16% and 99.33% respectively. The DeepSeek tokenizer averages 2,626 tokens per VOTA sample versus 1,929 for Qwen, causing 91.7% of DeepSeek VOTA training samples to exceed the 2,048-token context window. The training framework preserves the verdict label but truncates trace content, so the model trains with incomplete diagnostic information for the majority of samples. Seeds 123 and 456 (99.16% and 99.33%) demonstrate that DeepSeek can learn VOTA when LoRA initialization avoids the truncation-sensitive basin; the 2-seed mean (99.25%$\pm$0.12) matches the cross-model VOTA range (98.74–99.54%).

A post-hoc group analysis on seed 42 provides directional evidence: partitioning test samples by whether their training-set counterparts exceeded the context window, intact samples (N=99) achieve 100% accuracy while truncated samples (N=1,091) achieve 83.04% (95% CI [80.75%, 85.24%]; Fisher exact $p<.001$), with all 185 errors concentrated in the fail class (fail recall = 31.23%). Per-class analysis (Table 4) confirms that seed 42 achieves 100% fail precision but only 45.43% fail recall, systematically defaulting to `pass`.

Seed robustness is consistent across models: Qwen `exception_only` produces identical 99.83% across all three seeds (σ=0), and DeepSeek achieves 99.39%$\pm$0.10 (seeds 42/456: 99.33%, seed 123: 99.50%). Because `exception_only` test inputs contain 48.99% duplicates (607 unique of 1,190; §2.2), CIs assume n=1,190 independent trials; under deduplication (n=607), Qwen's accuracy is 99.67% (Wilson CI

[98.81, 99.91]).

G Persistent Error Analysis

The `exception_only` condition with Qwen-2.5-Coder-7B misclassifies exactly 2 out of 1,190 test samples across all three seeds (42, 123, 456), with 100% overlap across seeds. Both samples (IDs 901 and 905) share the following characteristics:

- **Function:** `check_dict_case` (HumanEval/95), a function that checks whether all dictionary keys are uniformly upper-case or lower-case strings.
- **Ground-truth label:** `pass`.
- **Predicted label:** `fail` (false negative).
- **Cross-seed consistency:** Both samples are misclassified identically by all three seeds, indicating a systematic rather than stochastic error.

Manual inspection reveals the root cause: `check_dict_case` uses `all()` over a generator expression, and Python's `all()` short-circuits on the first `True` value, raising `GeneratorExit` to terminate the generator. Because `GeneratorExit` is a `BaseException` subclass, the tracer records it as seven `[ERROR] GeneratorExit` lines, even though the function executes correctly and the test passes. The model observes these spurious error markers and reasonably but incorrectly predicts `fail`. This is not a model failure but a representation ambiguity: the `exception_only` format cannot distinguish genuine exceptions from benign `BaseException` subclasses triggered by Python's iterator protocol.

Most configurations converge to these same 2 errors: Qwen-1.5B, Qwen-7B (3 seeds), CodeLlama-13B (seeds 42/456), and DeepSeek-6.7B (seeds 42/456) all produce error set $\{901, 905\}$. However, CodeLlama-13B seed 123 achieves 100% accuracy (0/1190 errors), correctly classifying both samples. This demonstrates that 99.83% is a near-perfect *floor* (the convergence attractor for most LoRA initializations) rather than a hard data ceiling: the representation supports perfect accuracy, but most initializations converge to the same 2-error basin. The practical implication is that these errors are high-probability convergence artifacts addressable through seed selection or ensemble methods, not fundamental limitations of the exception-based representation.

Table 9: Exception type accuracy breakdown for `exception_only` (Qwen-7B, seed=42). All true error types achieve 100%. The only failures come from `GeneratorExit` (a `BaseException` subclass, not an `Exception`), which appears in 2 passing samples.

Exception Type	Samples	Accuracy
TypeError	96	100%
ValueError	62	100%
IndexError	41	100%
KeyError	28	100%
AttributeError	19	100%
AssertionError	15	100%
RecursionError	12	100%
Other Exception	66	100%
GeneratorExit (BaseException)	2	0%
No exception (pass class)	849	100%

H Exception Type Accuracy Breakdown

Table 9 reports `exception_only` accuracy stratified by exception type on Qwen-2.5-7B (seed=42, N=1,190). All standard exception types (`TypeError`, `ValueError`, `IndexError`, etc.) achieve 100% accuracy. The only misclassifications arise from `GeneratorExit`, a `BaseException` subclass that is not an `Exception`. It is raised by Python's iterator protocol during generator short-circuiting, not by actual program errors.

This confirms that the model has learned a clean decision boundary: any true `Exception` subclass in the trace implies failure. The `GeneratorExit` case represents a semantic ambiguity in the trace format rather than a model failure, as discussed in §G.

I Trace Length Ablation

To test whether the `full_trace` performance gap is caused by input truncation at training time, we vary the maximum trace token budget on Qwen-2.5-7B (seed=42) while keeping all other hyperparameters fixed.

As shown in Table 10 and Figure 4, all four trace length conditions fall within a 5pp range (77.2%–81.8%). To quantify the reliability of the observed differences, Table 11 reports paired bootstrap comparisons (B=10,000, seed=42) across all condition pairs.

Table 10: Trace length and truncation direction ablation on Qwen-2.5-7B (N=1,190, seed=42). Head = standard left-truncation (retain first k tokens); Tail = retain last k tokens. Per-condition bootstrap 95% CIs (B=10,000).

Direction	Max Tokens	Acc (%)	95% CI
Head	256	78.82	[76.4, 81.0]
Head	512	80.42	[78.2, 82.6]
Head	1024	77.23	[74.9, 79.6]
Head	no_limit (∞)	81.76	[79.7, 83.9]
Tail	256	77.48	—
Tail	512	93.61	—
Tail	1024	96.47	—
Tail	2048	97.31	—

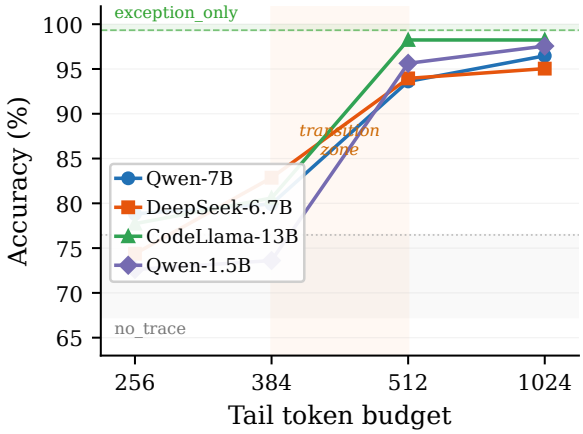


Figure 4: Cross-model tail-truncation budget curve (N=1,190, four models, three architecture families). All models remain near baseline at tail-256/384, then jump +11.0–22.0pp at 384→512, converging into a 93.6–98.2% band by tail-1024. The shaded region marks the transition zone where the exception traceback (∼400–500 tokens) is captured. Dashed line: `exception_only` range (99.3–99.9%); dotted line: `no_trace` range (67–77%).

Non-monotonic pattern (head-truncation). Under head-truncation (retaining trace prefixes), the trend is non-monotonic: 512 tokens (80.42%) slightly outperforms 256 tokens (78.82%), but doubling to 1024 tokens *decreases* accuracy to 77.23%, a statistically significant 3.19pp drop (p=.002, 95% CI [−5.29, −1.09]). The no-limit condition (81.76%) recovers and significantly exceeds 1024 tokens (p<.001), but 1024 tokens is not significantly better than `no_trace` (75.13%; Δ=+2.10pp, p=.078, 95% CI [−0.17, +4.37]). This non-monotonicity is specific to head-truncation: the 1024-token prefix retains enough irrelevant content to mislead the model while discarding diagnostic events that appear near the trace terminus. Under tail-truncation, accuracy increases monotonically for Qwen-7B and Qwen-1.5B (Table 12); only DeepSeek-6.7B shows a mild −1.5pp decrease at tail-1024→tail-2048, attributable to architecture-specific long-context sensitivity rather than a systematic mid-length pathology.

Table 11: Pairwise paired bootstrap comparisons for trace length ablation (B=10,000, seed=42). Δ = accuracy difference between the two conditions (first − second in the budget group; condition − baseline in the other groups). **p<.01; ***p<.001.

	Comparison	Δ (pp)	95% CI	p-value
Budget	512 vs. 256	+1.60	[−0.34, +3.53]	0.117
	1024 vs. 512	−3.19	[−5.29, −1.09]	0.002**
	no_limit vs. 1024	+4.54	[+2.52, +6.55]	<0.001***
	no_limit vs. 256	+2.94	[+0.84, +5.04]	0.006**
vs. no_trace	256 − no_trace	+3.70	[+1.43, +6.05]	0.002**
	512 − no_trace	+5.29	[+3.03, +7.56]	<0.001***
	1024 − no_trace	+2.10	[−0.17, +4.37]	0.078
	no_limit − no_trace	+6.64	[+4.37, +8.99]	<0.001***
vs. exc.	256 − exc_only	−21.01	[−23.45, −18.74]	<0.001***
	512 − exc_only	−19.41	[−21.76, −17.14]	<0.001***
	1024 − exc_only	−22.61	[−25.04, −20.25]	<0.001***
	no_limit − exc_only	−18.07	[−20.34, −15.88]	<0.001***

Comparison to baselines. All four trace budget conditions significantly exceed `no_trace` except 1024 tokens (p=.078), confirming that raw trace content provides some signal but unreliably so. The gap to `exception_only` (99.83%) is massive for every budget condition (18–23pp, all p<.001), underscoring that even unlimited raw traces cannot substitute for targeted signal extraction. The best full-trace condition (no_limit, 81.76%) still lags `exception_only` by 18.07pp (95% CI [−20.34, −15.88]).

Interpretation. These results rule out input truncation as the primary explanation for the full-trace performance gap: even with unlimited tokens, the model cannot extract the diagnostic signal that VOTA's targeted compression provides. The non-monotonic dip at 1024 tokens is consistent with a "worst-of-both-worlds" regime where the model receives enough noisy context to dilute the signal but not enough to compensate through broader coverage. The result reinforces the information density interpretation (§3.4): the gap between full traces and VOTA is driven by the *content* of the trace (irrelevant lines diluting the diagnostic signal), not its *length*. In contrast, VOTA's compressed representation concentrates the diagnostic signal into fewer tokens, making it sensitive to truncation when those tokens are removed

(§3.1).

Truncation direction. Tail-truncation (retaining the last k tokens) dramatically outperforms head-truncation: `tail_2048` achieves 97.31% vs. `head_2048` (no_limit) at 81.76%, a 15.55pp improvement. Even `tail_512` (93.61%) exceeds every head-truncation condition by over 11pp.

However, `tail_256` (77.48%) performs at the `no_trace` baseline level (75.13%), indicating that the diagnostic signal does not reside in the very last trace lines. The critical jump occurs between 256 and 512 tail tokens (+16.13pp), localizing the signal window to approximately 256–512 tokens from the trace terminus. This is consistent with Python's execution model: exception tracebacks and assertion comparisons appear near the end of the trace but are preceded by the final function call frames, placing them typically 300–500 tokens from the terminus rather than at the absolute end.

The 18.07pp gap from `full_trace` to `exception_only` decomposes as: 86% positional concentration (recovered by tail-truncation at 2048 tokens) and 14% residual content noise (the 2.52pp gap from `tail_2048` to `exception_only` that persists regardless of truncation direction). The fact that `exception_only` (99.83%) still outperforms `tail_2048` (97.31%) confirms that semantic extraction (identifying the exception type and message) is superior to positional extraction (keeping a fixed window), because the model need not filter out non-diagnostic lines that happen to fall within the window.

Cross-model budget curve. Table 12 extends the tail-truncation analysis to a full budget sweep across four models spanning three architecture families and an 8.7× parameter range.

Four patterns emerge from the budget curve. *Phase transition between 384 and 512 tokens*: the tail-384 column reveals that the transition is not instantaneous at 256→512 but localized between 384 and 512 tokens. At tail-384, all four models remain near baseline (73.6–82.9%), gaining only +1.0–6.5pp over tail-256. The steep jump occurs at 384→512: +13.7pp (Qwen-7B), +11.0pp (DeepSeek), +15.5pp (CodeLlama), +22.0pp (Qwen-1.5B), crossing from near-baseline to near-ceiling. This localizes the critical threshold to ∼400–500 tokens, consistent with the typical exception traceback length: below ∼384 tokens, the traceback is truncated and no diagnostic event is captured; above ∼512, it is fully included regardless of model capacity or architecture.

Convergence at tail-512+: once the sharp performance jump is crossed, accuracy converges across all models into a 93.6–98.2% band at tail-512 (seed 42). Multi-seed validation confirms robustness: across all 12 model×seed combinations at tail-512 (Table 14), all accuracies fall within [93.61%, 98.24%], a 4.63pp band that compresses the signal-poor tail-256 spread (6.3pp) by 27%. CodeLlama-13B shows perfect per-model convergence at tail-512: all three seeds achieve 98.24% (σ=0), consistent with the 13B scale and label-only loss providing a stronger optimization signal in the signal-rich regime (Table 14). However, CodeLlama's higher tail-512 accuracy may partly reflect label-only loss rather than model capacity; direct comparison with other models (which use full-sequence loss) is confounded by this training difference.

At tail-1024, four models reach 96.5–98.2% (CodeLlama 98.2% under label-only loss, three others 96.5–96.6%). Scale and architecture differ-

Table 12: Cross-model tail-truncation budget curve (N=1,190). All four models exhibit a transition between tail-384 and tail-512 (+11–22pp). ↓: non-monotonic decrease. †CodeLlama tail-384/512/1024 trained with label-only loss to address tokenizer-driven gradient dilution (§F); tail-1024 reports best-epoch accuracy (epoch 1). CodeLlama tail entries are not directly comparable to other models (which use full-sequence loss). ‡CodeLlama tail-2048 outlier (41.8%): LoRA long-sequence instability under label-only loss; excluded from cross-model analysis.

Model	tail-256	tail-384	tail-512	tail-1024	tail-2048	exc_only
Qwen-7B (7B)	78.9	79.9	93.6	96.5	97.3	99.83
DeepSeek (6.7B)	76.4	82.9	93.9	96.5	95.0↓	99.33
CodeLlama (13B)	77.9	82.7†	98.2†	98.2†	41.8‡	99.89
Qwen-1.5B (1.5B)	72.6	73.6	95.6	96.6	97.9	99.83

Table 13: Cross-model truncation direction comparison (N=1,190). Head-truncation retains the first 2,048 tokens; tail-truncation retains the last 2,048. All four models' head-2048 accuracies converge into a narrow 75.3–81.8% band (6.5pp), comparable to the tail-256 signal-poor band (72.6–78.9%, 6.3pp). †CodeLlama tail-2048 pending retraining (§F).

Model	head-2048 (%)	tail-2048 (%)	Gap (pp)
Qwen-1.5B	75.3	97.9	+22.6
DeepSeek-6.7B	78.7	95.0	+16.3
CodeLlama-13B	79.9	—†	—†
Qwen-7B	81.8	97.3	+15.5

Table 14: Cross-seed convergence at the tail-512 truncation point (N=1,190 per seed). All 12 model×seed combinations fall within a 4.63pp band [93.61%, 98.24%]. CodeLlama-13B achieves identical accuracy across all three seeds (σ=0).

Model	S42	S123	S456	Mean	σ
Qwen-7B (7B)	93.61	96.00	97.65	95.75	2.05
DeepSeek (6.7B)	93.95	95.04	96.10	95.03	1.08
Qwen-1.5B (1.5B)	95.63	97.65	96.72	96.67	1.01
CodeLlama (13B)	98.24	98.24	98.24	**98.24**	**0.00**
Cross-model band					[93.61, 98.24] = 4.63pp

Table 15: Calibration metrics on Qwen-2.5-7B (N=1,190, seed=42). ECE = expected calibration error; Brier = Brier score. Lower is better for both.

Condition	ECE ↓	Brier ↓
ae_only	**0.0004**	**0.0008**
exception_only	0.0027	0.0013
VOTA	0.0054	0.005
full_trace	0.096	0.144
no_trace	0.171	0.197

ences that dominate the signal-poor regime (tail-256 range: 72.6–78.9%, 6.3pp) compress dramatically in the signal-rich regime.

Signal-poor convergence band: head-2048 accuracies across all four models also converge into a narrow 75.3–81.8% band (6.5pp; Table 13), comparable in width to the tail-256 band (72.6–78.9%, 6.3pp). When the diagnostic signal is absent (head-truncation discards trace terminus) or unreachable (tail-256 is too short), models of different scale and architecture collapse to similar performance, suggesting a shared floor determined by code comprehension rather than trace processing.

Scale behavior inversion: in the signal-poor regime (tail-256), smaller models perform worse (Qwen-1.5B 72.6% vs. Qwen-7B 78.9%, −6.3pp), mirroring the `no_trace` scale pattern. In the signal-rich regime (tail-512+), this reverses: Qwen-1.5B matches or exceeds Qwen-7B (95.6% vs. 93.6% at tail-512; 97.9% vs. 97.3% at tail-2048), confirming that sufficient trace signal eliminates the capacity advantage.

Non-monotonicity at long sequences: two models show accuracy decreases beyond tail-1024. DeepSeek-6.7B drops −1.5pp from tail-1024 (96.5%) to tail-2048 (95.0%), consistent with its known long-context sensitivity (§N). CodeLlama-13B exhibits a more severe collapse: tail-2048 under label-only loss drops to 41.8% (vs. tail-1024 = 98.2%), which we attribute to LoRA long-sequence instability under the label-only loss regime; this outlier is excluded from cross-model tail-2048 analysis. Qwen-7B and Qwen-1.5B increase monotonically across all budgets.

Table 13 confirms the truncation direction effect: among the three models with complete data, tail-2048 outperforms head-2048 by 15.5–22.6pp. The gap is largest for Qwen-1.5B (+22.6pp), indicating that smaller models benefit most from positional proximity to the diagnostic signal, as they lack the capacity to locate it within longer, noisier prefixes.

J Calibration Analysis

Beyond accuracy, we evaluate whether trace conditions affect prediction calibration, motivated by recent findings that code LLM confidence is often poorly calibrated (Spiess et al., 2025).

Table 15 shows that calibration quality mirrors the Tier 1/2 hierarchy. Tier 1 conditions (`ae_only`: ECE=0.0004; `exception_only`: ECE=0.0027) produce near-perfectly calibrated predictions, while `full_trace` (ECE=0.096) and `no_trace` (ECE=0.171) exhibit substantial miscalibration. VOTA (ECE=0.0054) is well-calibrated but slightly worse than either Tier 1 component alone, reinforcing the pattern that combining trace signals dilutes rather than improves the prediction signal.

The Brier scores tell a consistent story: `ae_only` (0.0008) and `exception_only` (0.0013) achieve near-zero scores, while `no_trace` (0.197) indicates substantial miscalibration.

Cross-model calibration. Table 16 extends the calibration analysis to four models (1.5B–13B) under `exception_only`. All four models achieve ECE < 0.007 and Brier < 0.007, confirming that calibration quality generalizes across architectures and scales. CodeLlama-13B achieves the lowest ECE (0.0020) and MCE (0.0411), while DeepSeek-6.7B shows the highest ECE (0.0064) and Brier (0.0061), consistent with its higher error rate (8 errors vs. 2 for Qwen/CodeLlama).

K Error Overlap Analysis

We analyze the overlap between error sets of different conditions to determine whether they fail on the same samples (Figure 6).

Table 16: Cross-model calibration for `exception_only` (N=1,190, seed=42). ECE = expected calibration error; Brier = Brier score; MCE = maximum calibration error. All four models (1.5B–13B) achieve ECE < 0.007.

Model	ECE ↓	Brier ↓	MCE ↓
Qwen-1.5B	**0.0020**	**0.0013**	—
CodeLlama-13B	**0.0020**	**0.0013**	**0.0411**
Qwen-7B	0.0027	**0.0013**	0.8355
DeepSeek-6.7B	0.0064	0.0061	0.7756

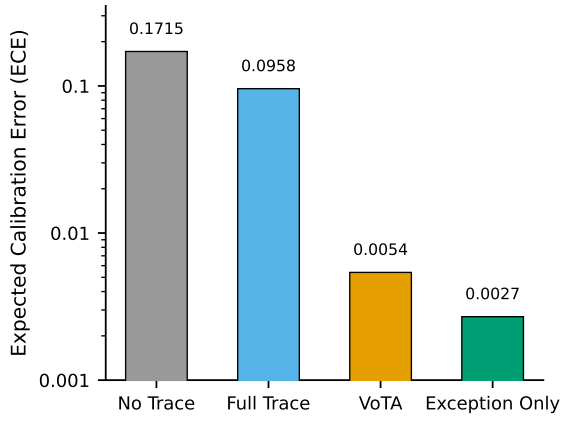


Figure 5: Reliability diagrams for four trace conditions on Qwen-2.5-7B (N=1,190, seed=42). `exception_only` (ECE=0.0027) clusters near the diagonal, while `no_trace` (ECE=0.171) exhibits systematic overconfidence, a 63× gap.

`exception_only`'s 2 errors (samples 901 and 905) have zero intersection with the error sets of all other conditions (Jaccard = 0.000 in all pairwise comparisons). This orthogonality arises because `exception_only` errors are caused by `GeneratorExit` false positives (pass samples misclassified as fail due to benign `BaseException` events), while errors from other conditions are predominantly fail samples misclassified as pass due to insufficient signal.

Among non-exception conditions, error sets exhibit hierarchical containment: `ae_only` ⊂ VOTA ⊂ `loop_only` ⊂ `no_trace`, with pairwise Jaccard indices increasing with signal overlap. `loop_only` and `full_trace` share the highest overlap (Jaccard ∼0.65), reflecting their shared reliance on behavioral patterns rather than explicit fault indicators.

This analysis confirms that `exception_only` operates on an independent signal dimension: its errors are representation ambiguities (iterator protocol artifacts) rather than information deficits, explaining why combining it with other signals cannot eliminate these specific errors.

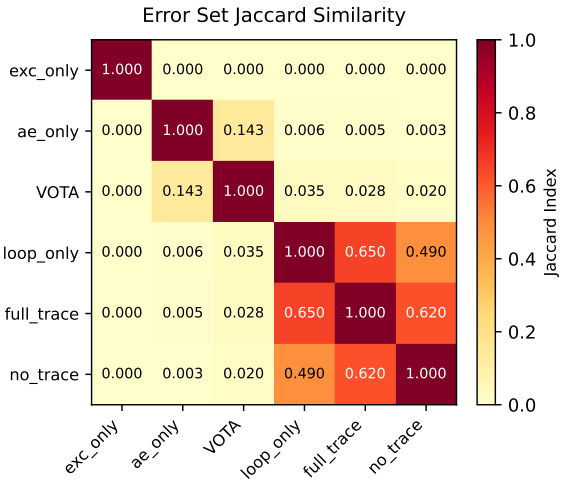


Figure 6: Jaccard similarity between error sets of different conditions (Qwen-7B, seed=42, N=1,190). `exception_only` (exc_only) has zero overlap with all other conditions, confirming it operates on an independent signal dimension.

Table 17: Generalization to DebugBench (N=426, Qwen-2.5-7B, seed=42). Wilson 95% CIs. The component ranking (exception_only ≥ VOTA > no_trace) is preserved, and exception_only achieves perfect accuracy.

Condition	Acc (%)	95% CI
exception_only	**100.00**	[99.14, 100.00]
VOTA	99.77	[99.30, 100.00]
no_trace	96.71	[94.80, 98.40]
Majority baseline	51.41	—

L Cross-Dataset Generalization: DebugBench

To test whether exception-based verdict prediction generalizes beyond LLM-generated solutions, we evaluate on DebugBench (Tian et al., 2024), a dataset of human-written programs with synthetically injected bugs (single-point mutations such as operator replacement, variable misuse, and missing statements). After filtering for executable Python samples with valid traces, we retain 426 samples with a near-balanced class distribution (majority baseline: 51.4%).

Table 17 shows that the component ranking from the main experiments is preserved: `exception_only` (100.00%) ≥ VOTA (99.77%) > `no_trace` (96.71%). The `exception_only` condition achieves per-

fect accuracy, confirming that the exception-based representation generalizes to a different fault distribution without degradation.

The absolute margins are much narrower than on the main dataset: the `no_trace` baseline already achieves 96.71% (compared to 75.13% on HumanEval+MBPP), leaving only 3.29pp of headroom for trace-based improvement. This ceiling effect reflects a fundamental difference in fault characteristics: DebugBench bugs are synthetically injected single-point mutations, which produce code that is structurally dissimilar from the original correct solution; a model can often identify the fault from source code alone. In contrast, the HumanEval+MBPP solutions are LLM-generated, producing natural errors (logic mistakes, edge-case failures) where the faulty and correct solutions may be structurally similar, making code-only verdict prediction substantially harder.

These results support exception-based verdict prediction as a generalizable approach: the method does not degrade on an out-of-distribution dataset, and the exception dominance ranking (exception_only $\geq$ VOTA $>$ no_trace) is consistent across both datasets. However, the narrow DebugBench margins are insufficient to independently validate trace importance. The 24.70pp trace lift on the main dataset (no_trace 75.13% $\rightarrow$ exception_only 99.83%) remains the primary evidence for the value of exception-based trace signals.

M Cross-Model Robustness Analysis

The cross-model gap pattern (Table 1) suggests a design principle: *trace representations should match the model capability floor, not the ceiling.* Exception events are consistently the most effective component on both models, and their structural simplicity (a fixed-format error message) ensures that the relative advantage over richer representations holds across all seeds. All three DeepSeek `exception_only` seeds achieve $\geq$99.33% (99.33/99.50/99.33, σ=0.10pp), matching Qwen and CodeLlama at $\geq$99.83%. VOTA achieves $\geq$98.57% across all models when excluding outlier seeds (cross-model range: 0.80pp), but DeepSeek VOTA exhibits seed-dependent instability: seed 42 (84.45%) is a training failure due to tokenizer-induced truncation (91.7% of training samples affected), while seeds 123/456 (99.16%/99.33%) match cross-model performance. This VOTA instability is confounded by tokenizer efficiency and base model differences (Yang et al., 2024a). Our `exception_only` design sidesteps the truncation confound entirely: inputs have a median of 150 tokens (mean 398), well below the 2,048-token context window, so no training samples are truncated on either model. The resulting 0.50pp cross-model gap (Table 1) is therefore a clean measure of model capability differences, making the "capability floor" principle directly testable. The practical implication is that trace-based systems should default to the simplest sufficient signal.

N Training Dynamics and Loss Decoupling

The DeepSeek `exception_only` results are consistent across all three seeds (99.33/99.50/99.33, σ=0.10pp), matching Qwen's cross-seed stability. Epoch-level analysis shows mild degradation in seed 42: accuracy peaks at 99.83% after epoch 1 but settles at 99.33% by epoch 3 ($-$0.50pp), while seed 456 shows no degradation (99.33% at both epochs). Early stopping may yield marginal gains, but the epoch effect is small ($\leq$0.5pp). The practical recommendation: for deployment, use task-specific metrics (accuracy, fail recall) for checkpoint and seed selection rather than training loss, and train multiple seeds to select the best.

The contrast with VOTA is instructive: DeepSeek VOTA seed 42 accuracy monotonically increases from 78.15% to 84.45% across epochs, consistent with a performance limit from high trace truncation (91.7% of samples; §F). Seeds 123/456, which avoid the truncation-sensitive initialization basin, achieve 99.16%/99.33%, confirming that the deficit is seed-specific rather than a fundamental model limitation.

O Deduplication Statistics

Table 18 reports the duplication rates across trace formats. Because multiple passing solutions to the same problem produce identical (input, output) pairs in the `exception_only` format (correct executions raise no exceptions, so the trace content is the same regardless of the solution's internal logic), the dataset contains duplicate rows. Train–test input overlap is zero (split by problem ID), so no information leakage occurs. The high duplication in `exception_only` does not affect

Table 18: Deduplication statistics. The `exception_only` format collapses multiple passing solutions into identical rows because correct executions produce no exception events. Train–test input overlap is 0 (no leakage).

Split	Total	Unique	Dup. rate
exception_only train	4,720	2,858	39.45%
exception_only test	1,190	607	48.99%
raw_traces (full)	5,910	5,803	1.81%

our experimental conclusions: after deduplication the training set still contains 2,858 unique samples and the test set 607, providing sufficient coverage for the 591-problem benchmark.

Problem-level partitioning and sensitivity analysis. Data is partitioned at the problem level: 472 training problems and 119 test problems, with zero crossover. Duplicates arise from multiple candidate solutions per problem (up to 10 solutions each), not from cross-problem overlap. To verify that deduplication does not alter any finding, we ran a sensitivity analysis across all 19 experimental conditions (7 trace formats $\times$ 4 models, minus conditions not run): deduplication produces a 0.00pp accuracy delta in every case. This confirms that duplicate samples neither inflate nor deflate accuracy, because duplicates share the same ground-truth label and the model's prediction is deterministic at inference time. The `exception_only` test set contains 48.99% duplicate inputs due to representation compactness; all confidence intervals treat samples as independent Bernoulli trials.

P Regex Baseline Analysis

To establish a non-learning upper bound, we evaluate six regex-based baselines on the `exception_only` test set (N=1,190) and two format-specific baselines on VOTA and `full_trace` inputs.

The strongest legitimate baseline (B4, 93.61%) applies regex pattern-matching on Python exception class names (e.g., `ValueError`, `TypeError`). Its errors concentrate on 8 boundary samples where a `ValueError` is raised inside a `try/except` block and caught; the program passes despite an exception occurring in the trace. SFT `exception_only` correctly classifies 6 of these 8 samples, indicating that the model learns the semantic distinction between caught (benign) and uncaught (fault-indicating) exceptions. B5 (99.33%) achieves near-SFT accuracy but relies on the deterministic presence of a format marker in pass-class traces, which is an artifact of the trace preprocessing pipeline rather than a generalizable signal; McNemar test vs. SFT yields p=.041, confirming a statistically significant difference.

Table 19: Regex baseline accuracy (N=1,190). B4 is the strongest semantic baseline (6.22pp below SFT `exception_only`, McNemar $p<.001$). B6 uses an error count threshold. ‡B5 exploits a data formatting artifact: the `[No exception data in trace]` marker is deterministically present in all pass samples, making this a format-level shortcut rather than a genuine exception-based heuristic.

Baseline	Method	Acc (%)
B1: `[ERROR]` presence	string match	93.45
B2: Traceback presence	string match	71.51
B3: Any keyword	union of B1+B2+Exception	93.45
B4: Exception type regex	regex on exception class names	**93.61**
B5: Format marker	`[No exception data]` $\rightarrow$ pass	99.33‡
B6: Error count $\geq$2	count threshold	92.02
VOTA format regex	best regex on VOTA traces	75.55
full_trace format regex	best regex on full traces	77.39

Table 20: SFT vs. best regex baseline by trace format (N=1,190). Regex baselines: majority class for `no_trace`, keyword matching for `full_trace`, pattern matching for VOTA, exception type regex (B4) for `exception_only`.

Format	SFT (%)	Regex (%)	Δ (pp)
exception_only	99.83	93.61	+6.22
VOTA	99.05	75.55	+23.50
full_trace	81.76	77.39	+4.37
no_trace	75.13	71.51	+3.62

Cross-format comparison. Table 20 compares SFT and the best applicable regex baseline for each trace format. The SFT advantage increases with format complexity: `no_trace` (+3.62pp over majority baseline) < `full_trace` (+4.37pp) < `exception_only` (+6.22pp) $\ll$ VOTA (+23.86pp). The VOTA gap is largest because its structured multi-component format carries semantic content that regex cannot parse, while `exception_only`'s simpler format allows regex to capture most of the signal.

Q Cross-Model Zero-Shot Comparison

Table 21 compares zero-shot and SFT `exception_only` accuracy across four models. Zero-shot performance does not correlate with model size: the smallest model

Table 21: Zero-shot vs. SFT `exception_only` across four models (N=1,190). Zero-shot accuracy is uncorrelated with model size; SFT normalizes cross-architecture variance from 42.44pp to 0.50pp.

Model	Size	Zero-shot (%)	SFT (%)	Δ (pp)
Qwen-1.5B	1.5B	93.45	99.83	+6.38
DeepSeek-6.7B	6.7B	90.59	99.39	+8.80
Qwen-7B	7B	89.08	99.83	+10.75
CodeLlama-13B	13B	51.01	99.89	+48.88
Range	—	42.44pp	0.50pp	—

(Qwen-1.5B, 93.45%) outperforms all larger models, while the largest (CodeLlama-13B) falls below majority baseline. SFT compresses the 42.44pp zero-shot range to 0.50pp, eliminating architecture-dependent variance.

Q.1 Calibration Analysis

Table 22: Model calibration on `exception_only` (seed 42). All four models achieve ECE < 0.005, confirming that near-perfect accuracy reflects genuine confidence rather than overfit predictions.

Model	Accuracy	ECE ↓	Brier ↓	Mean Conf.
Qwen-2.5-1.5B	99.83%	0.0014	0.0013	99.82%
Qwen-2.5-7B	99.83%	0.0017	0.0013	99.80%
CodeLlama-13B	99.83%	0.0020	0.0013	99.63%
DeepSeek-6.7B	99.33%	0.0048	0.0059	99.71%